운영 장애 분석 실습 기록

MemoryGuard · CPU Watchdog · Deadlock

codyssey_work_B4-2

실험일: 2026-10-09 / 시간대: KST(UTC+09:00)

실행 계정: jae0608(UID 1000) / Ubuntu 26.04.1 LTS / WSL2 / x86_64

수록 내용

1. OOM Crash Issue 보고서
2. CPU Latency Issue 보고서
3. Deadlock Issue 보고서
4. 명령어·옵션·실제 출력 상세 해설
5. 공통 Issue 템플릿
6. 선택 과제: 응용 작업 스케줄러 분석

검증 개요

OOM: 128 → 256MB에서 로그 기준 생존 시간 17.202 → 32.363초.
CPU: 80 → 40 설정에서 구간 최대 50.0 → 40.0%, 55초 생존 확인.
Deadlock: true의 순환 락 대기·정체와 false의 작업 진행 비교.

앱 내부 지표와 OS 측정값, 평균과 구간 값, 보호 종료와 수동 종료를 구분합니다. 바이너리 디컴파일·역공학은 수행하지 않았습니다.

실제 실행 7건의 로그·CSV·스냅샷과 SHA256SUMS는 저장소 evidence/에 함께 보관했습니다.

[Bug] OOM Crash - MEMORY_LIMIT 초과 시 MemoryGuard 자체 종료

실험일: 2026-10-09 KST · 실행 계정: jae0608(UID 1000) · 환경: Ubuntu 26.04.1 LTS / WSL2 / x86_64

1. Description (현상 설명)

agent-leak-app을 일반 사용자로 실행하면 부팅 검사 6개가 모두 통과하지만, MemoryWorker의 메모리 값이 약 3초마다 25MB씩 증가하고 MemoryGuard 로그와 함께 종료됐습니다.

Before는 MEMORY_LIMIT=128, After는 256입니다. CPU_MAX_OCCUPY=100과 MULTI_THREAD_ENABLE=false를 동일하게 유지해 **메모리 설정 하나만 변경**했습니다.

항목	Before	After
설정	128 / 100 / false	256 / 100 / false
실제 작업 PID	962	1156
시작 로그	11:32:59.967	11:37:40.809
MemoryGuard 종료 요청	11:33:17.169	11:38:13.172
로그 기준 생존 시간	17.202초	32.363초
앱 내부 종료 시 메모리	150MB	275MB
최초 관측 RSS	18,356KiB (17.926MiB)	18,476KiB (18.043MiB)
마지막 관측 RSS	146,376KiB (142.945MiB)	274,516KiB (268.082MiB)
종료 코드	137	137

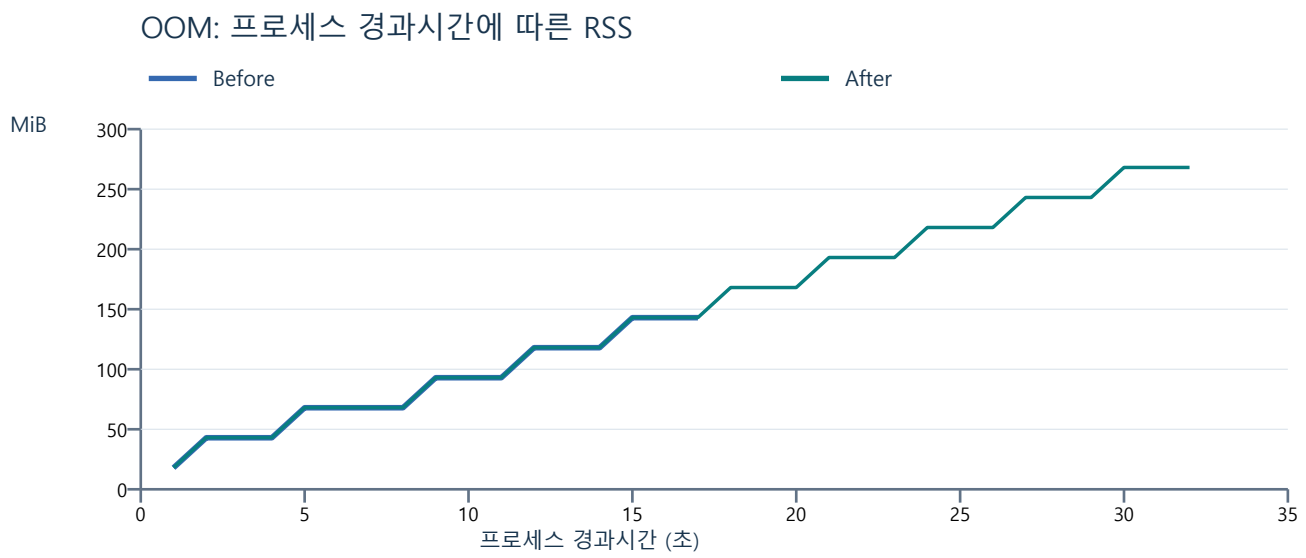
2. Evidence & Logs (증거 자료)

재현 경로

1. 필수 환경변수·디렉터리·키 파일을 준비하고 15034 포트가 비어 있는지 확인했습니다.
2. source ./env.sh로 초기 값을 적용해 128MB 케이스를 실행했습니다.
3. 부모 PID와 실제 작업을 수행하는 자식 PID를 각각 monitor.sh로 기록했습니다.
4. 종료 후 PID가 사라진 것을 ps -C agent-leak-app으로 확인하고 wait의 종료 코드를 저장했습니다.
5. export MEMORY_LIMIT=256만 변경하고 새 증거 폴더에 같은 절차를 반복했습니다.

```
source ./env.sh
export MEMORY_LIMIT=128        # After에서는 256으로 변경
export CPU_MAX_OCCUPY=100
export MULTI_THREAD_ENABLE=false
./agent-leak-app > "$RUN_DIR/app.log" 2>&1 &
APP_PID=$!
./monitor.sh "$APP_PID" > "$RUN_DIR/monitor-$APP_PID.csv" &
# pgrep -P "$APP_PID"로 자식 PID도 찾아 각각 관제했다.
```

관제 자료



한도128/256MB의 초기 곡선 일부는 겹침. 앱 내부 Heap와 RSS는 별개 지표.

OOM RSS 비교

128MB 케이스의 실제 CSV 발췌입니다. CPU 값은 ps의 누적 평균이며 RSS의 단위는 KiB입니다.

```
timestamp,pid,ppid,elapsed_s,cpu_avg_pct,mem_pct,rss_kib,threads,state
2026-10-09T11:33:01+09:00,962,954,1,2.0,0.2,18356,1,SN
2026-10-09T11:33:05+09:00,962,954,5,1.1,0.8,69564,1,SN
2026-10-09T11:33:11+09:00,962,954,12,0.9,1.5,120772,1,SN
2026-10-09T11:33:17+09:00,962,954,17,0.8,1.8,146376,1,SN
```

종료 로그

Before:

```
2026-10-09 11:33:17,169 [INFO] [MemoryWorker] Current Heap: 150MB
2026-10-09 11:33:17,169 [CRITICAL] [MemoryGuard] Memory limit exceeded (150MB >= 128MB) / (Recommend Over 256MB)
2026-10-09 11:33:17,169 [CRITICAL] [MemoryGuard] Self-terminating process 962 to prevent system instability.
```

After:

```
2026-10-09 11:38:13,171 [INFO] [MemoryWorker] Current Heap: 275MB
2026-10-09 11:38:13,172 [CRITICAL] [MemoryGuard] Memory limit exceeded (275MB >= 256MB) / (Recommend Over 256MB)
2026-10-09 11:38:13,172 [CRITICAL] [MemoryGuard] Self-terminating process 1156 to prevent system instability.
```

두 실험 모두 종료 후 ps 출력에는 다음 제목 행만 남았습니다.

```
PID PPID ELAPSED %CPU %MEM RSS NLWP STAT
```

원본: 128MB 로그, 128MB CSV, 256MB 로그, 256MB CSV. 설정·PID·종료 후 출력·종료 코드도 각 폴더에 보존했습니다.

3. Root Cause Analysis (원인 분석)

증거로 확인한 원인

메모리 증가와 종료가 두 번 재현됐고, 앱이 직접 Memory limit exceeded와 Self-terminating process를 기록했습니다. 따라서 **직접적인 종료 원인은 앱 내부 MemoryGuard 보호 정책**으로 판단합니다.

앱의 메모리 값은 25MB 단위로 증가하므로 128MB를 정확히 찍지 않고 125 → 150MB, 256MB는 250 → 275MB 단계에서 초과했습니다. 설정 범위를 벗어난 입력이나 부팅 실패가 아닙니다.

관측 기간에 누적 데이터를 회수하지 않는 메모리 누수 또는 데이터 보관 증가가 의심됩니다. 다만 비공개 바이너리만으로 특정 컬렉션이나 코드의 해제 누락 위치, 도달 불가능 객체의 발생을 확정하지 않았습니다.

OS 동작 원리와 측정 구분

프로세스 주소 공간에는 코드·정적 데이터·힙·스택 등이 있습니다. 동적으로 생성하는 데이터의 증가가 힙 및 실제 상주 페이지의 증가로 나타날 수 있습니다. RSS는 물리 메모리에 상주하는 프로세스 메모리이며, 앱 내부 Current Heap과 같은 지표가 아닙니다. ps 문서

1초 간격 관제는 마지막 25MB 할당 직후의 짧은 구간을 놓칠 수 있습니다. **142.945/268.082MiB는 관측 최대 RSS이지 실제 종료 순간의 절대 최대값이 아닙니다.**

종료 코드 137은 Bash의 128+시그널 번호 규칙에서 SIGKILL(9) 해석과 일치합니다. 종료 주체는 코드 숫자만이 아니라 MemoryGuard 로그로 판단했습니다. 커널 로그로 OOM Killer 발생을 입증한 실험은 아닙니다. Bash 종료 상태 문서

누수가 지속되면 다른 프로세스에 줄 물리 메모리가 줄고, 페이지 회수·스왑이 늘어 시스템 지연으로 이어질 수 있습니다. 이번 종료는 전체 RAM 소진 전에 앱 설정 한도에서 발생했습니다.

4. Workaround & Verification (조치 및 검증)

MEMORY_LIMIT을 **128 → 256MB**로 상향했습니다. 생존 시간이 **17.202 → 32.363초**, 약 **15.161초 증가(1.88배)**했습니다.

그러나 두 번째 실행도 메모리가 증가하다 같은 정책으로 종료됐습니다. 이번 비교에서 한도 상향은 **종료를 지연한 임시 조치**입니다.

추가 제안은 데이터 보관량 제한, 캐시 만료·회수 정책, 작업 완료 후 참조 제거 및 소스 접근이 가능한 개발자의 할당 분석입니다. 실제 정상 시나리오(512 / 40 / false)에서는 별도 실험에서 Memory Cache Flushed와 MEMORY RECOVERED도 관측됐습니다. 여러 설정이 다른 별도 시나리오이므로 이를 이번 단일 변수 비교의 결과와 섞지 않습니다.

[Bug] CPU Latency - 부하 증가 후 Watchdog 종료, CPU_MAX_OCCUPY=40으로 회피

실험일: 2026-10-09 KST · 환경: Ubuntu 26.04.1 LTS / WSL2 / x86_64

1. Description (현상 설명)

MEMORY_LIMIT=512, MULTI_THREAD_ENABLE=false에서 CPU 상한 설정을 80으로 실행하자 CpuWorker의 내부 부하 값이 증가한 뒤 Watchdog가 SIGTERM 보호 종료를 수행했습니다.

비교 실행은 CPU_MAX_OCCUPY만 **80 → 40**으로 바꿨습니다. 두 비교 실행 모두 PTY 출력 기록과 top -d 0.1을 사용했습니다.

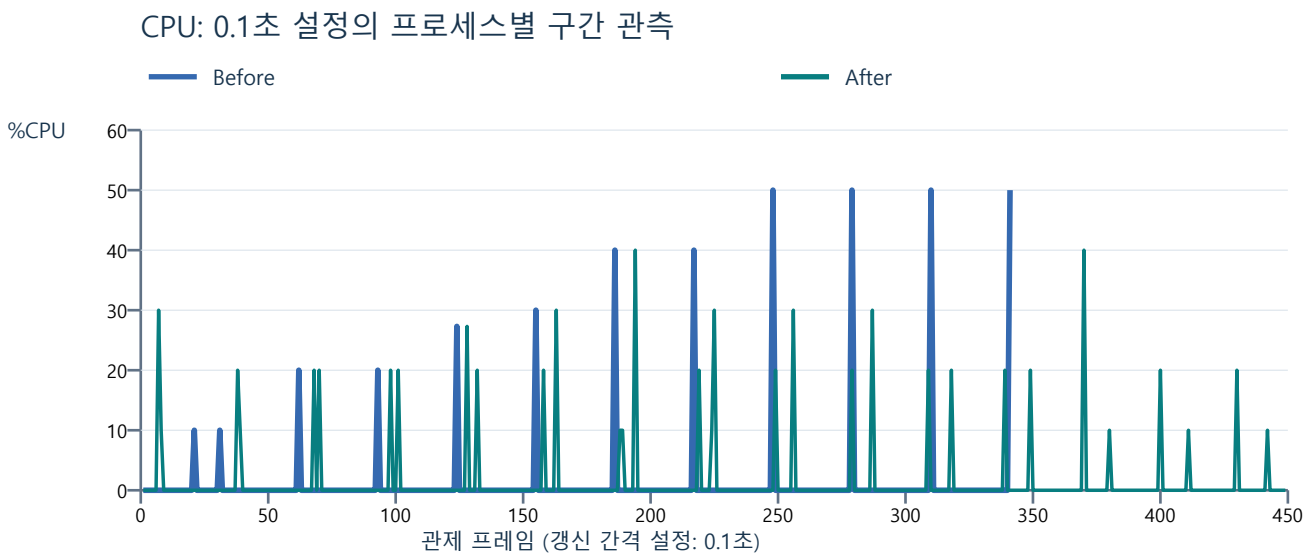
항목	Before: 80	After: 40
작업 PID	1921	2359
나머지 설정	512 / false	512 / false
OS 구간 CPU 관측 최대	50.0%	40.0%
앱 내부 지표	53.11%에서 임계치 위반	40.00%에서 냉각 시작
종료/생존	시작 로그부터 36.380초 후 Watchdog 종료	55초 시점 PID·로그 진행 확인
최종 종료 코드	143, Watchdog SIGTERM	143, 실험자 수동 SIGTERM

최대값은 첫 top 프레임을 제외한 결과입니다. Before는 앱이 살아 있는 341개 구간 표본, After는 449개 표본입니다. 50%가 지속됐거나 노트북 전체 CPU의 50%였다는 의미가 아닙니다.

2. Evidence & Logs (증거 자료)

재현·수집 명령

```
source ./env.sh
export MEMORY_LIMIT=512
export CPU_MAX_OCCUPY=80        # After는 40
export MULTI_THREAD_ENABLE=false
script -q -e -f -c './agent-leak-app' "$RUN_DIR/app.log" ₩
  < /dev/null > "$RUN_DIR/console.log" 2>&1 &
SESSION_PID=$!
# 앱 PID는 pgrep -x agent-leak-app으로 별도로 찾았다.
TOP_PIDS=$(pgrep -x agent-leak-app | paste -sd, -)
LC_ALL=C top -b -d 0.1 -n 450 -w 180 -p "$TOP_PIDS" ₩
  > "$RUN_DIR/top.log" 2> "$RUN_DIR/top-error.log" &
```



첫 프레임 제외. Before는 종료 이후 관측값 없음. CPU 0으로 채우지 않음.

CPU 구간 관측 비교

Before: 앱 로그

```
2026-10-09 11:57:44,269 [INFO] Agent listening at port 15034
2026-10-09 11:57:46,280 [INFO] [CpuWorker] Current Load: 5.00%
2026-10-09 11:58:20,549 [INFO] [CpuWorker] Current Load: 53.11%
2026-10-09 11:58:20,649 [CRITICAL] [CpuWorker] CPU Threshold Violated! (53.11%).
>>> [SYSTEM] WATCHDOG: INITIATING EMERGENCY ABORT (SIGTERM) <<<
Script done on 2026-10-09 11:58:20+09:00 [COMMAND_EXIT_CODE="143"]
```

top.log에서 PID 1921의 0.1초 구간 최대 50.0%를 확인했고, 종료 후 ps-check.txt는 제목 행만 남았습니다.

After: 냉각·생존 로그

```
2026-10-09 12:21:08,526 [INFO] [CpuWorker] Peak reached (40.00%). Starting cooldown...
2026-10-09 12:21:33,450 [INFO] [CpuWorker] Cooldown complete (5.00%). Resuming load increase...

2026-10-09T12:21:40+09:00
PID  PPID ELAPSED %CPU %MEM RSS    NLWP STAT
2359 2358 55      1.4  6.1  479420 3    SNl+
```

표의 행은 원본 ps-check.txt의 열을 읽기 쉽게 정렬한 것입니다. %CPU=1.4는 누적 평균이며, top의 구간 최대 40.0%와 측정 기간이 다릅니다.

추가 관측이 필요했던 이유

초기 80 설정의 1초 간격·30회 top에서는 최대 4.0%가 기록됐고, 기록은 11:48:00에 끝났습니다. 실제 종료는 11:48:05.646이었습니다. 짧은 버스트와 종료 직전 구간을 확인하기 위해 같은 80 설정으로 0.1초·450회 기록을 추가했습니다.

직접 파일로 출력한 초기 로그에는 Watchdog 문구가 없었지만, PTY로 재수집하자 SIGTERM 문구까지 확인됐습니다. Python 표준 출력의 터미널·파일 버퍼링 방식 차이와 일치하는 결과입니다. 버퍼링만이 유일한 원인이었다고 확정하지는 않았습니다. Python 출력 설명

원본: Before 로그, Before top, After 로그, After top, After 생존 확인.

3. Root Cause Analysis (원인 분석)

직접적인 종료 원인은 앱의 CpuWorker 임계치 위반과 **Watchdog의 SIGTERM 보호 조치**입니다. 내부 Current Load가 올라가는 동작과 실제 프로세스의 짧은 CPU 상승이 함께 관측됐습니다.

CPU_MAX_OCCUPY=80인데 내부 지표 53.11%에서 종료했으므로, 이 환경변수를 그대로 "OS CPU 사용률 80%를 넘으면 종료하는 임계치"로 해석하지 않습니다. 관측상 40 설정은 내부 부하 증가 상한·냉각 동작을 제어했고, 정확한 내부 보호 임계치 구현은 확인하지 않았습니다.

CPU를 사용하는 실행 가능 작업은 같은 CPU의 다른 작업과 실행 시간을 경쟁합니다. 버스트나 실행 대기 증가가 응답 시간을 늘릴 수 있습니다. 다만 이 실험은 **프로세스 CPU 상승과 보호 종료**를 검증했으며, 요청 지연 시간·P99 지연이나 시스템 전체 CPU 포화를 측정하지 않았습니다.

ps %CPU는 실행 이후 평균, top %CPU는 갱신 구간 기준입니다. 앱 내부 계산 값은 또 다른 지표입니다. 서로 대체해 적지 않았습니다. ps, top

4. Workaround & Verification (조치 및 검증)

CPU_MAX_OCCUPY를 **80 → 40**으로 낮췄습니다. After는 내부 40%에서 냉각됐고, 동일한 구간 관제의 최대값이 50.0 → 40.0%로 낮아졌습니다. Before 종료 시간보다 긴 **55초 시점에 PID와 진행 로그가 유지**됐습니다.

After 전체 로그에는 Watchdog/CPU 임계치 위반이 없고 마지막 종료는 cleanup.log의 수동 SIGTERM입니다. 두 실행 모두 종료 코드가 143이어도 원인은 다릅니다. 코드 숫자와 종료 로그·수동 조치 기록을 함께 판단합니다.

After의 장기 기록에는 벽시계와 elapsed_s 사이에 공백/차이가 있어, "약26분 동안 중단 없이 실행"처럼 확장해 주장하지 않습니다. 보고서의 기본 비교 범위는 명확한 초기 55초입니다.

근본 조치 제안은 CPU 작업량·동시성 제한, 배치 크기 및 휴지 구간 조절, 별도 부하/지연 벤치마크와 프로파일링입니다.

[Bug] Deadlock - 상반된 락 획득 순서로 두 워커가 순환 대기

실험일: 2026-10-09 KST · 환경: Ubuntu 26.04.1 LTS / WSL2 / x86_64

1. Description (현상 설명)

MEMORY_LIMIT=512, CPU_MAX_OCCUPY=40을 유지하고 MULTI_THREAD_ENABLE=true로 실행하자 프로세스는 살아 있지만 로그와 메모리가 정체됐습니다. 두 워커는 각자 자원 하나를 보유하고 상대 자원을 기다렸습니다.

MULTI_THREAD_ENABLE만 false로 바꾼 After에서는 PID가 유지되고 작업·로그가 계속 진행됐습니다.

항목	Before: true	After: false
실제 작업 PID	9226	11294
두 관측 시각	12:59:55 / 13:00:05	13:03:21 / 13:03:31
PID 존재	두 시점 존재	두 시점 존재
RSS	18,512 → 18,512KiB	121,000 → 197,812KiB
로그 크기	2,792 → 2,792바이트	3,474 → 3,870바이트
스레드	세 스레드 모두 futex_do_wait	메인 futex, 두 워커 poll 대기; 로그 진행
최종 종료	수동 SIGTERM	수동 SIGTERM

2. Evidence & Logs (증거 자료)

재현·관측 명령

```
source ./env.sh
export MEMORY_LIMIT=512
export CPU_MAX_OCCUPY=40
export MULTI_THREAD_ENABLE=true   # After는 false
script -q -e -f -c './agent-leak-app' "$RUN_DIR/app.log" ₩
  < /dev/null > "$RUN_DIR/console.log" 2>&1 &
SESSION_PID=$!

ps -ef | grep '[a]gent-leak-app'
ps -L -p "$WORKER_PID" -o pid,tid,pcpu,rss,time,stat,wchan:32,comm
stat -c 'log_bytes=%s modified=%y' "$RUN_DIR/app.log"
# 같은 명령을 10초 간격으로 두 번 기록했다.
```

PID·스레드 대기 증거

13:00:05 원본 스냅샷의 작업 프로세스입니다.

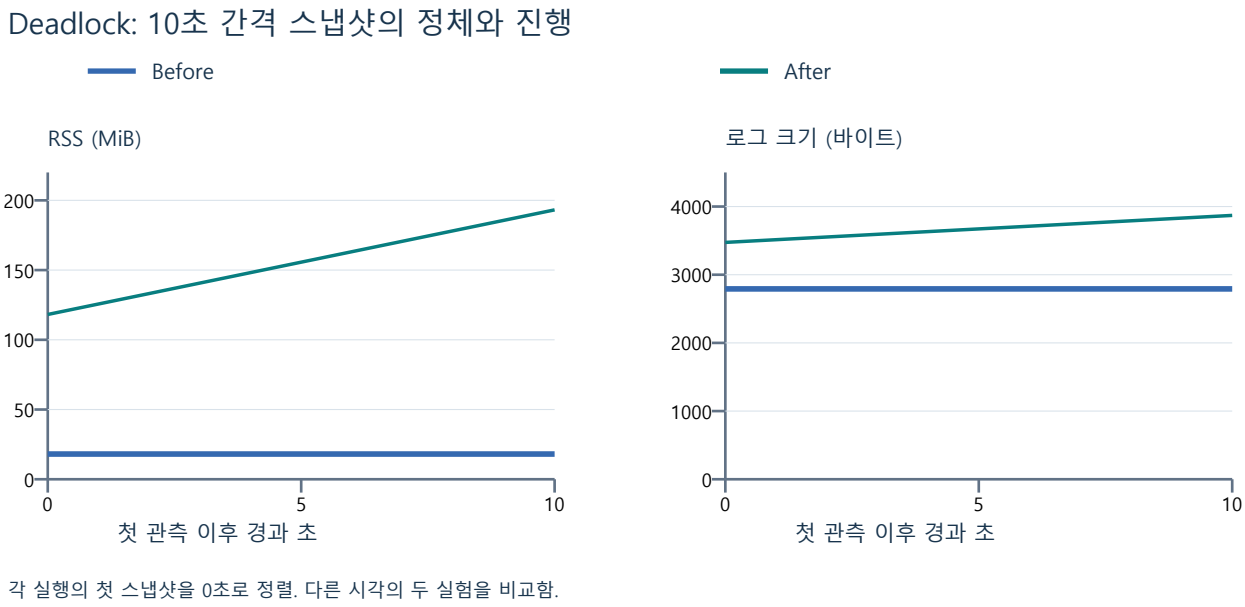
```
jae0608 9226 9225 0 12:59 pts/2 00:00:00 ./agent-leak-app
PID  TID  %CPU RSS   TIME     STAT WCHAN         COMMAND
9226 9226 0.1  18512 00:00:00 SNl+ futex_do_wait agent-leak-app
9226 9297 0.0  18512 00:00:00 SNl+ futex_do_wait agent-leak-app
9226 9298 0.0  18512 00:00:00 SNl+ futex_do_wait agent-leak-app
log_bytes=2792 modified=2026-10-09 12:59:52.286312863 +0900
```

동일한 RSS를 세 행만큼 합산하지 않습니다. 각 TID는 같은 프로세스의 주소 공간을 공유합니다. 로그의 Worker-Thread-1/2와 TID9297/9298의 개별 대응 정보가 없으므로 어느 TID가 어느 워커인지 확정하지 않았습니다.

마지막 로그와 순환 대기

```
12:59:50,274 Worker-Thread-1 LOCK ACQUIRED: [Shared_Memory_A]. (Holding...)
12:59:50,275 Worker-Thread-2 LOCK ACQUIRED: [Socket_Pool_B]. (Holding...)
12:59:52,284 Worker-Thread-2 WAITING for [Shared_Memory_A]... (Status: BLOCKED)
12:59:52,286 Worker-Thread-1 WAITING for [Socket_Pool_B]... (Status: BLOCKED)
```

이 발췌는 원본에서 날짜와 공통 로그 접두사를 생략한 것입니다. 마지막 앱 로그는 12:59:52.286이며 두 스냅샷 사이에 파일 크기와 mtime이 변하지 않았습니다.



Deadlock 스냅샷 비교

원본: Before 로그, 스냅샷1, 스냅샷2, Before CSV, After 스냅샷1, After 스냅샷2.

3. Root Cause Analysis (원인 분석)

대기 관계는 **Thread-1 → B(Thread-2 보유) → Thread-2 → A(Thread-1 보유) → Thread-1**입니다. 두 워커의 락 획득 순서가 반대이므로 한쪽이 자원을 반납하지 않으면 다른 쪽도 진행하지 못합니다. 메인도 워커 완료를 기다리는 로그와 대기 상태가 관측됐습니다.

교착상태 조건	정의	이번 관측과의 연결
상호 배제	자원을 한 작업만 소유	A/B 각각의 LOCK ACQUIRED
점유 대기	보유한 채 다른 자원을 기다림	Holding 뒤 상대 자원 WAITING
비선점	대기자가 다른 작업의 락을 강제 회수하지 못함	대기 중 락 해제·자동 회수 없음; 일반 락 모델로 추론
순환 대기	대기 관계가 고리를 형성	T1은 B, T2는 A를 기다림

futex_do_wait는 커널 동기화 대기 상태와 부합합니다. 다만 futex 대기만으로 교착을 단정하지 않았습니다. **보유·추가 요청 로그, 순환 대기 관계, 진행 정체**를 함께 근거로 진단했습니다. 락 구현 종류·타임아웃 코드는 확인하지 않았습니다. futex 문서

4. Workaround & Verification (조치 및 검증)

MULTI_THREAD_ENABLE=true → false로 변경하고 동일한 두 스냅샷 수집을 반복했습니다. After에서는 로그 크기가 **396 바이트 증가**, RSS가 **76,812KiB 증가**했고 워커 진행 로그가 계속 발생했습니다. 관측한 10초 구간에서 해당 교착 경로가 회피됐습니다.

After에서도 메인 스레드의 futex 대기와 스레드 3개가 남았습니다. 정상 워커도 타이머·이벤트를 기다릴 수 있습니다. 설정 false를 프로그램 전체의 단일 스레드 전환으로 해석하지 않고, 로그와 워커 진행으로 회피를 확인했습니다.

두 실행은 증거 수집 후 수동 SIGTERM으로 정리했습니다. 원래 Deadlock은 프로세스가 스스로 종료된 장애가 아닙니다. 마지막 exit-code.txt=143은 정리 행위의 결과이며 cleanup.log가 이를 구분합니다.

근본 해결 제안은 모든 경로에서 A → B처럼 락 획득 순서 통일, 불필요한 중첩 락 제거, 타임아웃·롤백, 예외 발생 시 락 해제 보장입니다.

명령어·실행 목적·실제 출력 해석 상세 기록

실제로 수행한 작업을 **입력 명령 → 목적·옵션 → 실제 출력 → 판단 → 과제 요구사항** 순서로 설명합니다. 반복 실행은 공통 절차로 묶고, 케이스별 설정·PID는 마지막 표에서 추적합니다. 예상 예시를 실제 측정값처럼 기록하지 않았습니다.

1. 환경과 사전 준비

실험일은 2026-10-09 KST, 환경은 Ubuntu 26.04.1 LTS / WSL2 / x86_64입니다. 최종 nproc는14, free -h는 Linux RAM 7.5GiB와 swap 2.0GiB를 표시했습니다. 환경 원본

Windows의 RAM, WSL에 보이는 RAM, 앱의 MEMORY_LIMIT은 서로 다른 값입니다. MEMORY_LIMIT은 애플리케이션에 전달한 보호 정책 설정입니다.

관리자 PowerShell에서:

```
wsl --install -d Ubuntu
```

--install은 WSL·배포판을 설치하고 -d Ubuntu는 배포판을 선택합니다. 실제로 WSL3.0.1과 Ubuntu 설치가 완료됐습니다. Ubuntu 일반 계정 jae0608을 생성했습니다. 비밀번호는 기록하지 않습니다.

Ubuntu 창에서:

```
cd ~
whoami
id -u
uname -r
uname -m
```

명령	목적	실제 출력의 의미
cd ~	현재 사용자의 Linux 홈으로 이동	Windows System32에서 /home/jae0608로 이동
whoami	사용자 이름 확인	jae0608
id -u	숫자 UID 확인	1000. root의 UID0이 아님
uname -r	커널 확인	6.18.40.1-microsoft-standard-WSL2
uname -m	CPU 아키텍처 확인	x86_64

jae0608@HP:~$는 입력할 코드가 아니라 프롬프트입니다. 사용자·호스트·현재 위치를 표시합니다. root가 아닌 계정 확인은 과제의 첫 실행 조건을 충족하는 증거입니다.

2. 제공 파일 선택과 실행 권한

```
file /mnt/c/Users/1234v/Downloads/agent-app-leak/agent-leak-app-*
mkdir -p ~/jae-test
cp /mnt/c/Users/1234v/Downloads/agent-app-leak/agent-leak-app-x86 \
  ~/jae-test/agent-leak-app
chmod u+x ~/jae-test/agent-leak-app
ls -lh ~/jae-test/agent-leak-app
```

명령·표시	목적과 의미
/mnt/c	WSL에서 Windows C드라이브에 접근하는 경로
file	실행 파일 형식과 CPU 종류 확인. 코드 분석을 하지 않음
*	이름에 맞는 파일들을 셸이 선택
ELF64 x86-64	Linux64비트 x86 파일. 이번 머신과 호환되는 파일 형식

명령·표시	목적과 의미
ARM aarch64	ARM용 파일. 이번 실행에서는 x86 파일을 선택
dynamically linked	동적 로더·라이브러리를 사용하는 파일 형식
mkdir -p	폴더 생성. 이미 존재해도 그 이유로 실패하지 않음
cp	Windows 다운로드 파일을 Linux 작업 폴더에 복사
chmod u+x	소유자에게 실행 권한 추가
ls -l / -h	권한·소유자 등 상세 표시 / 사람이 읽기 쉬운 크기

실제 출력:

```
-rwxr-xr-x 1 jae0608 jae0608 6.3M Oct 9 09:27 /home/jae0608/jae-test/agent-leak-app
```

첫 -는 일반 파일입니다. 뒤9글자는 소유자·그룹·기타의 r 읽기, w 쓰기, x 실행 권한입니다. 소유자 jae0608에게 실행 권한이 있음을 확인했습니다.

3. 환경변수·필수 디렉터리·키 파일

실제로 저장한 설정 파일:

```
export AGENT_HOME="$HOME/jae-test"
export AGENT_PORT=15034
export AGENT_UPLOAD_DIR="$AGENT_HOME/upload_files"
export AGENT_KEY_PATH="$AGENT_HOME/api_keys"
export AGENT_LOG_DIR="$AGENT_HOME/logs"
export MEMORY_LIMIT=128
export CPU_MAX_OCCUPY=100
export MULTI_THREAD_ENABLE=false
```

파일 작성에는 아래 형태의 here-document를 사용했습니다.

```
cat > ~/jae-test/env.sh <<'EOF'
# 이 사이에 위 export 명령들을 넣었다.
EOF
```

cat이 EOF까지의 입력을 파일에 씁니다. 'EOF'처럼 구분자를 따옴표로 감싸면 파일을 만드는 시점에 $HOME을 확장하지 않고 글자 그대로 보관합니다. 이후 source할 때 해당 사용자의 값으로 확장됩니다.

설정 적용:

```
source ~/jae-test/env.sh
mkdir -p "$AGENT_UPLOAD_DIR" "$AGENT_KEY_PATH" "$AGENT_LOG_DIR"
printf '%s' 'agent_api_key_test' > "$AGENT_KEY_PATH/secret.key"
chmod 600 "$AGENT_KEY_PATH/secret.key"
```

- source는 현재 셸에서 설정을 실행합니다. 파일을 저장한 것만으로 환경변수가 적용되지는 않습니다.

- export는 자식 프로그램에도 값을 전달합니다.

- $HOME은 사용자 홈이며, 큰따옴표는 경로를 하나의 인자로 전달합니다.

- printf '%s'는 문자열을 쓰며 개행을 추가하지 않습니다. 실제 key 파일은18바이트였습니다.

- >는 출력 파일을 생성하거나 기존 내용을 덮어씁니다.

- 600은 소유자 읽기·쓰기만 허용합니다.

검증 명령:

```
printenv AGENT_HOME AGENT_PORT AGENT_UPLOAD_DIR AGENT_KEY_PATH AGENT_LOG_DIR \
  MEMORY_LIMIT CPU_MAX_OCCUPY MULTI_THREAD_ENABLE
ls -ld "$AGENT_UPLOAD_DIR" "$AGENT_KEY_PATH" "$AGENT_LOG_DIR"
```

```
ls -l "$AGENT_KEY_PATH/secret.key"
```

printenv는 요청한 순서로 값만 표시했습니다. 실제 값은 홈 경로,15034,세 디렉터리 경로,128,100,false였습니다. ls -d는 디렉터리 자체를 표시합니다. 세 폴더의 소유자와 쓰기 권한, key 파일의 -rw-------·18바이트를 확인했습니다.

과제 범위는 MEMORY_LIMIT 50~512 정수, CPU_MAX_OCCUPY10~100 정수, MULTI_THREAD_ENABLE 참/거짓입니다. 설정 파일은 초기 실험값이며 다음 케이스에서는 export로 덮어썼습니다.

4. 포트와 부팅 출력

```
ss -ltn 'sport = :15034'
```

ss는 소켓 상태 조회입니다. -l은 대기 중인 소켓, -t는 TCP, -n은 숫자 표시입니다. 필터는 로컬 포트15034만 선택합니다. 실제로 제목 행만 출력돼 현재 해당 TCP 포트에 대기 서비스가 없음을 확인했습니다.

앱 부팅 로그는 계정·환경변수·파일·포트·로그 권한·미션 설정의 6개 검사에 모두 [OK], 마지막에 All Boot Checks Passed! / Agent READY를 표시했습니다.

Recommend Over 256MB / Recommend Under 50%는 권장값 안내였습니다. 부팅 검사는 통과했으므로 이를 부팅 실패로 기록하지 않았습니다.

5. 증거 폴더와 백그라운드 실행

```
RUN_DIR="$AGENT_HOME/evidence/oom-128-$(date +%Y%m%d-%H%M%S)"
mkdir -p "$RUN_DIR"
printf 'MEMORY_LIMIT=%s\nCPU_MAX_OCCUPY=%s\nMULTI_THREAD_ENABLE=%s\n' \
  "$MEMORY_LIMIT" "$CPU_MAX_OCCUPY" "$MULTI_THREAD_ENABLE" \
  > "$RUN_DIR/settings.txt"
date -Iseconds > "$RUN_DIR/started_at.txt"
./agent-leak-app > "$RUN_DIR/app.log" 2>&1 &
APP_PID=$!
```

구문	의미
RUN_DIR=	이번 실행의 증거 경로 저장
$(date ...)	명령 출력으로 날짜·시간을 경로에 포함
%Y%m%d-%H%M%S	연월일-시분초 형식
settings.txt	실행 당시 환경변수의 실제 값 보존
date -Iseconds	시간대가 포함된 ISO 시각
./	현재 폴더의 실행 파일 지정
> app.log	표준 출력(FD1) 저장
2>&1	오류 출력(FD2)을 현재 표준 출력과 같은 곳에 연결
&	백그라운드 실행. 다음 명령 입력 가능
$!	직전 백그라운드 작업의 PID
[1] 954	작업 번호1, 시작 PID954라는 셸 알림

출력 연결은 왼쪽부터 적용하므로 > file 2>&1 순서로 사용했습니다. 새로운 증거 경로가 실험 간 덮어쓰기를 막습니다.

실제로 실행 파일은 부모와 작업 자식을 만들었습니다. 부모만 관제하면 자식의 메모리 상승을 놓칠 수 있어 둘 다 관제했습니다.

```
pgrep -P "$APP_PID"
pgrep -x agent-leak-app
```

-P는 지정 부모의 자식을, -x는 정확한 이름을 찾습니다. 일반 실행의 APP_PID와 PTY 실행의 SESSION_PID는 의미가 다릅니다.

6. 변수별 역할

변수	역할
AGENT_HOME	필수 파일·로그·실행 파일의 기준 경로
RUN_DIR	한 번의 실행 증거 폴더
APP_PID	일반 실행에서 시작한 앱 부모 PID
SESSION_PID	script로 시작한 PTY 세션 PID
APP_PARENT_PID	별도로 찾아낸 앱 부모 PID
WORKER_PID	실제 작업 자식 PID
PID / CHILD_PID	반복문에서 현재 관제·정리하는 PID
TOP_PIDS	top에 전달할 쉼표로 연결된 PID 목록
BEFORE_DIR 등	비교 전 증거 폴더를 보존하는 경로 변수

PID·경로를 launcher.pid, session.pid, worker.pid, observed-pids.txt 등에도 저장했습니다. 셸 변수가 없어져도 원본 파일을 추적할 수 있습니다.

7. monitor.sh: 코드의 목적과 CSV 해석

제공 폴더에는 monitor.sh가 없어 직접 작성했습니다. 사용한 원문

1. LC_ALL=C로 숫자·출력 형식을 통일합니다.
2. 인자가 하나이고 양의 정수 PID인지 검사합니다. $#는 인자 수, $1은 첫 인자입니다.
3. CSV 열 이름을 출력합니다.
4. ps -p "$pid"로 대상 PID만 조회합니다.
5. -o pid=,...로 필요한 필드를 선택하고 =로 ps 제목을 숨깁니다.
6. awk는 열을 쉼표로 연결하고 좀비 상태 Z를 제외합니다.
7. 결과가 비면 break로 끝냅니다.
8. 날짜와 값을 출력하고 sleep 1 후 반복합니다.

```
timestamp,pid,ppid,elapsed_s,cpu_avg_pct,mem_pct,rss_kib,threads,state
```

열	의미와 주의
timestamp	실제 수집 시각·시간대
pid / ppid	프로세스 / 부모 ID
elapsed_s	프로세스 경과 초
cpu_avg_pct	ps의 실행 이후 평균 CPU. 순간 값 아님
mem_pct	RSS를 Linux 전체 물리 메모리로 나눈 비율

열	의미와 주의
rss_kib	물리 메모리 상주량. KiB
threads	NLWP, 스레드 수
state	상태·부가 플래그

조회·파일 쓰기 시간이 들기 때문에 표본은 정확히 매1.000초가 아닙니다. 실제 수집 시각을 사용합니다. 단위 환산은 MiB=KiB/1024입니다. 146376KiB는142.945MiB입니다. 앱 내부 MB 표기와 RSS를 동일한 측정값으로 간주하지 않습니다.

S는 대기/수면, N은 낮은 우선순위, l은 멀티스레드, s는 세션 리더, +는 터미널 전경 프로세스 그룹 표시입니다. 상태 플래그 하나만으로 장애를 판정하지 않습니다.

8. OOM 명령과 실제 판단

```
source ./env.sh
export MEMORY_LIMIT=256
ps -C agent-leak-app -o pid,ppid,etimes,pcpu,pmem,rss,nlwp,stat
tail -n 80 "$RUN_DIR/app.log"
tail -n 20 "$RUN_DIR/monitor-962.csv"
```

ps -C는 이름으로 프로세스를 선택하고 -o는 출력 열을 선택합니다. tail -n N은 마지막 N줄입니다. CSV 이름의 PID는 실제 케이스에 맞춰야 합니다.

실제 Before Heap는25→50→75→100→125→150MB였고 150MB >= 128MB에서 종료했습니다. After는275MB에서 256MB를 초과했습니다. 한 번에25MB씩 증가하므로 설정값을 정확히 찍지 않을 수 있습니다.

RSS의 마지막 관측142.945MiB와 앱 종료 값150MB는 측정 대상·기본 런타임 메모리·간격이 달라 일치할 필요가 없습니다. 마지막 할당 직후의 강제 종료는 관제가 놓칠 수 있습니다.

%MEM=1.8의 분모는 앱 한도128MB가 아니라 Linux 전체 RAM입니다. 전체 RAM이 거의 찬 상태가 아니어도 앱 정책 한도를 넘으면 종료할 수 있습니다. 과제 예시의96%를 실제 값처럼 복사하지 않았습니다.

종료 후 ps 제목만 남은 것은 조회 시점에 이름이 일치하는 PID가 없다는 뜻입니다. MemoryGuard 로그와 함께 종료를 확인했습니다. 한도 상향으로17.202→32.363초 생존했지만 상승과 종료가 계속돼 임시 조치로 분류합니다.

9. 종료 코드 저장과 의미

```
wait "$APP_PID"
printf '%s\n' "$?" > "$RUN_DIR/exit-code.txt"
```

wait는 현재 셸의 자식 작업 종료를 기다리고 상태를 반환합니다. $?는 직전 명령의 종료 상태입니다. 다른 명령을 먼저 실행하면 값이 바뀌므로 바로 저장했습니다.

기록	해석
137	Bash128+9와 일치. MemoryGuard 로그와 결합해 판단
143 + WATCHDOG/SIGTERM	CPU 보호 종료
143 + cleanup.log	실험자 수동 종료
PID 존재 + 정체	종료와 다름. Deadlock에서는 락·스레드 근거 추가 필요

Killed와 Terminated는 셸 알림입니다. 코드나 알림만으로 커널 OOM·Watchdog를 확정하지 않았습니다. Bash 문서

10. CPU: PTY·짧은 간격 관제가 필요한 이유

초기1초 관제 최대는4%였지만 앱 내부 값은53.01%였습니다. 기록 종료가11:48:00, 앱 종료가11:48:05.646이라 마지막 구간도 빠졌습니다. 이 차이를 숨기지 않고 추가 관측했습니다.

```
script -q -e -f -c './agent-leak-app' "$RUN_DIR/app.log" \
  < /dev/null > "$RUN_DIR/console.log" 2>&1 &
SESSION_PID=$!
TOP_PIDS=$(pgrep -x agent-leak-app | paste -sd, -)
LC_ALL=C top -b -d 0.1 -n 450 -w 180 -p "$TOP_PIDS" \
  > "$RUN_DIR/top.log" 2> "$RUN_DIR/top-error.log" &
```

옵션	목적
script	가상 터미널에서 실행하고 출력 기록
-q / -e / -f	콘솔 안내 축소 / 자식 종료 코드 반영 / 자주 flush
-c	실행 명령 지정
< /dev/null	백그라운드에서 사용자 터미널 입력을 읽지 않게 연결
console.log	script의 출력·오류도 별도 보존
paste -sd, -	PID를 쉼표 목록으로 연결
top -b	파일 저장용 배치 모드
-d 0.1	갱신 간격0.1초 설정
-n 450	최대450회 후 top 종료
-w 180	출력 폭 지정
-p	해당 앱 PID만 조회

Python 표준 출력은 터미널에서 줄 버퍼링, 일반 파일에서는 보통 블록 버퍼링합니다. 강제 종료 직전 print가 남지 않을 가능성을 PTY로 보완했고 실제 Watchdog 문구를 확보했습니다. 버퍼링만이 유일한 원인이라고 확정하지는 않았습니다. Python 출력, script

Script started ... <not executed on terminal>은 script 입력이 사용자 터미널이 아니었다는 안내입니다. 앱이 실행되지 않았다는 뜻이 아닙니다.

top 최초 프레임은 안정적인 이전 갱신 구간이 없어 제외했습니다. 이후 최대는 PID1921에서50.0%, 비교 PID2359에서 40.0%였습니다. 일반적인 top 기본 표시는 논리 CPU 하나의100%를 기준으로 하므로 전체 CPU 요약과 구분합니다. 실제 응답 지연·전체 CPU 포화는 측정하지 않았습니다. top 문서

11. CPU 최대값 집계 코드

```
awk '
/^top -/ {frame++; next}
frame > 1 && $1 ~ /^[0-9]+$/ && $NF == "agent-leak-app" {
    p=$1
    c=$9+0
    if (!(p in peak) || c > peak[p]) peak[p]=c
    n[p]++
}
END {
    for (p in peak)
        printf "PID=%s MAX_CPU=%.1f%% SAMPLES=%d\n", p, peak[p], n[p]
}' "$RUN_DIR/top.log"
```

/^top -/는 화면 갱신을 세고 frame>1은 첫 프레임을 제외합니다. $1은 기본 출력의 PID, $9는 %CPU, $NF는 마지막 명령 열입니다. 숫자 PID와 앱 이름이 맞는 행만 선택합니다.

+0은 숫자 변환, peak[p]는 PID별 최대값, n[p]는 기록 수입니다. END에서 요약합니다. 사용자 정의 top 열 순서가 바뀌면 9번째 열이라는 가정도 수정해야 합니다.

실제 Before의 부모1920 최대0%, 작업1921 최대50%, 각341표본입니다. After의 부모2358 최대0%, 작업2359 최대40%, 각449표본입니다. Before 종료 뒤의 PID 없음 구간을 CPU0으로 채우지 않았습니다.

12. CPU 조치의 검증 범위

80 설정은 내부5→53.11% 뒤 Watchdog SIGTERM, 40 설정은 Peak reached (40.00%). Starting cooldown...과 Cooldown complete (5.00%)를 기록했습니다. After는55초 시점에 PID·진행 로그가 유지됐습니다.

환경변수80을 "OS CPU80%를 넘으면 종료"로 가정하지 않았습니다. 종료 시 내부 값은53.11%, 외부 구간 최대는50%였습니다. 앱 내부 지표, ps 평균, top 구간 값을 각각 보고했습니다.

정상 시나리오는 Heap525MB에서 Starting cleanup / Memory Cache Flushed / MEMORY RECOVERED도 기록했습니다. 초기 상승만으로 전체 정상 시나리오를 누수라고 판정하지 않았습니다.

13. Deadlock 스냅샷 함수와 옵션

```
capture_agent_state() {
  date -Iseconds
  ps -ef | grep '[a]gent-leak-app'
  ps -L -p "$WORKER_PID" \
    -o pid,tid,pcpu,rss,time,stat,wchan:32,comm
  stat -c 'log_bytes=%s modified=%y' "$RUN_DIR/app.log"
  tail -n 20 "$RUN_DIR/app.log"
}
sleep 8
capture_agent_state | tee "$RUN_DIR/snapshot-1.txt"
sleep 10
capture_agent_state | tee "$RUN_DIR/snapshot-2.txt"
```

구문·필드	의미
함수	같은 조회를 반복해 비교 조건 유지
ps -e / -f	전체 선택 / UID·PID·PPID 등 상세 형식
grep '[a]...'	앱 이름 검색, grep 패턴 자체는 동일 이름으로 매치되지 않게 작성
-L / -p	스레드 단위 / 작업 PID 선택
tid	스레드 ID. 메인 TID는 PID와 동일
time	누적 CPU 시간
wchan:32	커널 대기 위치 이름을 폭32로 표시
stat -c	파일 크기·mtime을 지정 형식으로 조회
%s / %y	파일 바이트 크기 / 마지막 수정 시각
sleep	지정한 초 동안 기다림
tee	화면과 파일에 동시에 출력

ps·stat·tail은 순차 수집해 완전히 같은 순간의 원자적 스냅샷이 아닙니다. 10초 사이 진행 여부를 비교했습니다.

14. Deadlock 출력의 의미

Before(512/40/true)는 PID9226 유지, RSS18512KiB 고정, 세 TID 모두 futex_do_wait, 로그2792바이트·mtime 동일입니다.

T1은 A를 보유하며 B를 기다리고 T2는 B를 보유하며 A를 기다렸습니다. 이는 순환 대기입니다. futex 대기만이 아니라 락 보유·추가 요청·BLOCKED 로그와 진행 정체를 결합했습니다.

After(512/40/false)에서는 메인 futex 대기가 남아도 두 워커가 poll_schedule_timeout.constprop.였고 로그3474→3870바이트, Heap·CPU 로그가 진행됐습니다. 낮은 CPU나 대기 상태 자체만으로 장애를 판정하지 않습니다.

false에서도 스레드3개입니다. 이 설정은 교착 유발 경로를 회피한 것으로 분석하며 프로세스 전체의 단일 스레드 전환이라고 적지 않습니다. 스레드 행의 RSS는 공유 프로세스 값이므로 세 행을 더하지 않습니다. Worker 이름과 각 TID의 일대일 대응도 미확인입니다.

15. 수동 정리와 백업

```
for PID in $(cat "$RUN_DIR/observed-pids.txt"); do
  if [[ "$(ps -p "$PID" -o comm= 2>/dev/null)" == "agent-leak-app" ]]; then
    printf '%s manual SIGTERM PID=%s\n' "$(date -Iseconds)" "$PID" \
      >> "$RUN_DIR/cleanup.log"
    kill -TERM "$PID" 2>/dev/null
  fi
done
wait "$SESSION_PID"
printf '%s\n' "$?" > "$RUN_DIR/exit-code.txt"
```

저장한 PID만 순회하며 현재 이름이 맞는지 확인합니다. kill -TERM은 SIGTERM 전달, >>는 덧붙이기, 2>/dev/null은 이미 종료된 PID의 오류 출력 생략입니다. 수동 요청 시각을 남겨 자동 Watchdog와 구분했습니다.

```
BACKUP_DIR="/mnt/c/Users/1234v/Downloads/agent-app-leak/results"
mkdir -p "$BACKUP_DIR/evidence"
cp -r "$AGENT_HOME/evidence/." "$BACKUP_DIR/evidence/"
cp "$AGENT_HOME/env.sh" "$AGENT_HOME/monitor.sh" "$BACKUP_DIR/"
{
  date -Iseconds
  uname -a
  id
  cat /etc/os-release
  nproc
  free -h
} > "$BACKUP_DIR/environment.txt"
```

cp -r는 하위 폴더 복사, evidence/.는 내용을 복사해 중첩 폴더를 피하는 형태입니다. { … }는 출력을 묶어 하나의 파일에 저장합니다. uname은 커널, id는 계정·그룹, os-release는 배포판, nproc는 CPU 수, free는 메모리·스왑을 기록합니다.

16. 실제 실행과 과제 요구사항 연결

폴더	메모리/CPU/멀티	작업 PID	목적
oom-128-20261009-113259	128/100/false	962	메모리 상승·보호 종료
oom-256-20261009-113740	256/100/false	1156	한도 변경·생존 시간 비교
cpu-80-20261009-114729	512/80/false	1530	초기 관측의 한계 확인
cpu-80-pty-20261009-115744	512/80/false	1921	급상승·Watchdog 증거
cpu-40-pty-20261009-122044	512/40/false	2359	CPU 상한·냉각 비교
deadlock-true-20261009-125943	512/40/true	9226	PID·락 순환·정체
deadlock-false-20261009-130309	512/40/false	11294	로그·작업 진행 비교

각 보고서는 현상·증거·원인·조치의 필수4개 절을 갖추었습니다. MEMORY_LIMIT은2회, CPU_MAX_OCCUPY는동일 관측법 2회, MULTI_THREAD_ENABLE은true/false2회로 비교했습니다.

17. 사실·추론·미확인

- 사실: 원본 PID·설정·로그·RSS·CPU 구간값·종료 코드·수동 조치.
- 추론: 낮은 메모리 설정의 회수 부족, 락 순서로 인한 순환 대기, 출력 버퍼링 영향.
- 미확인: 바이너리 코드 위치·정확한 보호 구현·실제 요청 지연·TID별 Worker 대응.
- 한계: 표본 사이 순간 최대를 놓칠 수 있고 장기 벽시계에는 공백이 있어 무중단 가동 시간을 확대해 주장하지 않음.

18. 터미널 종료 시 복구

파일로 저장한 로그·CSV·스크립트는 남습니다. 새 Ubuntu 셸에서 cd ~/jae-test와 source ./env.sh로 경로를 복원할 수 있지만 RUN_DIR·PID 변수·함수는 다시 지정해야 합니다.

새 셸은 이전 작업의 부모가 아니므로 wait로 과거 종료 코드를 임의로 복원하지 않습니다. 이미 저장된 exit-code와 로그를 확인합니다. 실행 중인 프로세스는 ps로 현재 상태를 확인하고 이전 PID 숫자만으로 종료하지 않습니다.

19. 참고 문서

- WSL 설치
- ps
- top
- script
- Bash 종료 상태
- Python 출력
- futex

[Bug] {장애 유형} - {한 줄 요약}

실험/발생 일시: {날짜·시각·시간대} 환경: {OS·커널·아키텍처·사용자 UID·자원 제한} 실행 폴더/PID: {폴더·부모 PID·작업 PID} Before / After 설정: {모든 설정을 적고 변경 변수 하나를 표시}

1. Description (현상 설명)

- 누가·어디서·언제·어떤 조건으로 실행했는가?
- 무엇이 관측됐는가? 종료/살아 있지만 정체/응답 지연을 구분한다.
- 재현 빈도와 관측 시간을 기록한다.

2. Evidence & Logs (증거 자료)

재현 경로

실제 입력한 준비·실행·관제 명령

각 명령의 목적과 주요 옵션을 설명하고 예상 출력이 아닌 실제 출력을 첨부한다.

관제 및 실행 로그

{날짜·PID·단위가 있는 원본 발췌}

- 원본 파일 링크와 그래프를 첨부한다.
- Heap/RSS, 앱 내부 CPU/OS CPU, 평균/구간 사용률을 구분한다.
- 종료 코드, PID 존재 여부, 수동 종료 기록을 함께 보존한다.

3. Root Cause Analysis (원인 분석)

- 확인한 사실: {로그·수치로 입증한 사항}
- 추론: {원인 가설과 근거}
- 확인하지 못한 사항: {소스 위치·정확한 타임아웃 등}
- 관련 OS 원리: {메모리/스케줄링/락 대기}

4. Workaround & Verification (조치 및 검증)

변경 변수: {이름, 기존 값 → 변경 값} 고정 변수: {비교에서 유지한 값}

항목	Before	After	근거 파일
{메모리/CPU/생존/로그 진행}	{값·단위}	{값·단위}	{링크}

- 관측 범위에서 무엇이 개선됐는가?
- 임시 조치인지 근본 해결인지 구분한다.
- 정상 실행을 수동 종료했다면 자동 장애 종료와 구분한다.
- 추가 제안: {소스 수정·추가 측정}

케이스별 제출 전 확인

- [ ] OOM: RSS 상승 CSV, MemoryGuard 종료 로그, MEMORY_LIMIT 최소 2회 비교
- [ ] CPU: 프로세스 CPU 상승 구간, WATCHDOG/SIGTERM, CPU_MAX_OCCUPY 비교
- [ ] Deadlock: PID 존재, ps -L/top -H, WAITING/BLOCKED, 락 순환 관계, true/false 비교
- [ ] 원본·설정·시각·단위·수동 종료 여부가 추적 가능함

[Analysis] 정상 로그에서 관측한 응용 작업 스케줄러의 순차 실행

1. 로그 관찰 개요

정상 시나리오의 Scheduler가 작업 목록 ['Thread-A', 'Thread-B', 'Thread-C']를 등록했습니다. 아래 분석은 응용 작업 로그에 대한 추론이며 Linux 커널 스케줄러 식별이 아닙니다.

2. 증거 자료

작업	시작	완료
Thread-A	12:20:46.925	12:20:47.127
Thread-B	12:20:47.178	12:20:47.381
Thread-C	12:20:47.431	12:20:47.634

각 작업은20→40→60→80→100%로 완료한 뒤 다음 작업이 시작했습니다. 전체 완료는12:20:47.685입니다. 원본

3. 패턴 분석 및 결론

관측한 배치에서는 등록 순서대로 완료하는 **FCFS형 순차 패턴**이 가장 단순한 설명입니다. 작업 중간의 A/B/C 교대 로그는 없었습니다.

우선순위 정보가 없어 같은 순서를 만드는 Priority 구현을 배제할 수 없고, 작업보다 긴 시간 할당량도 순차 완료처럼 보일 수 있습니다. 따라서 로그 기반 추론이며 OS 알고리즘이나 내부 코드를 확정하지 않습니다. Thread라는 이름과 실제 TID의 일대일 관계도 미확인입니다.

4. 장단점과 적용 범위

순차 처리 모델은 구현과 실행 순서 추적이 간단하고 순서가 중요한 배치에 적합합니다. 앞의 긴 작업이 뒤의 짧은 작업을 지연시킬 수 있습니다.

실시간 요청 처리에서는 작업 분할·동시성·공정성·우선순위와 실제 지연 측정이 필요합니다. 이번 결과를 근거로 Linux가 FCFS/Round-Robin/Priority 중 하나로 스레드를 처리한다고 단정하지 않습니다.